Django Practicals with Answers

Web Framework | Python | GTU Diploma Engineering

Covers: Project Setup · MVT · Models · Views · Templates · Forms · Admin · URL Routing · Static Files · Authentication · ORM Queries · CRUD Operations

Practical 1: Django Project Setup & App Creation

Aim: Install Django, create a project and an app, run the development server.

Step-by-Step Commands:

```
# Step 1: Install Django
pip install django

# Step 2: Check Django version
python -m django --version

# Step 3: Create a new project
django-admin startproject myproject

# Step 4: Navigate into project folder
cd myproject

# Step 5: Create an app inside the project
python manage.py startapp myapp

# Step 6: Run the development server
python manage.py runserver

# Visit: http://127.0.0.1:8000/
```

Project Structure:

```
myproject/
■■■ manage.py              # Command-line utility
■■■ myproject/
■   ■■■ __init__.py
■   ■■■ settings.py        # Project settings
■   ■■■ urls.py            # Root URL configuration
■   ■■■ asgi.py
■   ■■■ wsgi.py
■■■ myapp/
    ■■■ __init__.py
    ■■■ admin.py           # Admin registrations
    ■■■ apps.py            # App config
    ■■■ models.py          # Database models
    ■■■ views.py           # View functions/classes
    ■■■ urls.py            # App URL patterns
    ■■■ tests.py
    ■■■ migrations/        # Database migration files
```

Register App in settings.py:

```
# myproject/settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp',           # <-- Add your app here
]
```

Explanation:

django-admin startproject: creates the project with settings, urls, wsgi. **startapp**: creates a reusable module with models, views, urls. **manage.py runserver**: starts local dev server at port 8000. Always register the app in **INSTALLED_APPS** for Django to recognize it.

Output/Result: Browser shows the Django welcome page: 'The install worked successfully! Congratulations!'

Practical 2: MVT Architecture — First 'Hello World' Page

Aim: Understand Django's MVT (Model-View-Template) pattern by creating a simple Hello World page.

1. Create View (myapp/views.py):

```
from django.shortcuts import render
from django.http import HttpResponse

# Simple function-based view
def hello_world(request):
    return HttpResponse("<h1>Hello, World from Django!</h1>")

# View using a template
def home(request):
    context = {
        'title': 'Welcome to Django',
        'message': 'This is my first Django page!',
        'items': ['Python', 'Django', 'HTML', 'CSS'],
    }
    return render(request, 'myapp/home.html', context)
```

2. Create URLs (myapp/urls.py):

```
from django.urls import path
from . import views

urlpatterns = [
    path('',          views.home,        name='home'),
    path('hello/',    views.hello_world,  name='hello'),
]
```

3. Include App URLs in Project (myproject/urls.py):

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('',       include('myapp.urls')),  # include app urls
]
```

4. Create Template (myapp/templates/myapp/home.html):

```
<!DOCTYPE html>
<html>
<head>
    <title>{{ title }}</title>
</head>
<body>
    <h1>{{ title }}</h1>
    <p>{{ message }}</p>

    <h2>Technologies:</h2>
```

```
    <ul>
        {% for item in items %}
            <li>{{ item }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

5. Configure Templates in settings.py:

```
TEMPLATES = [{
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [],
    'APP_DIRS': True,   # looks inside app/templates/ folders
    ...
}]
```

Explanation:

Model: handles data (database). **View**: business logic, receives request, returns response. **Template**: HTML with Django Template Language (DTL). **{{ variable }}**: outputs variable. **{% tag %}**: logic tags (for, if, block). **render()**: renders template with context dict and returns HttpResponse. **context**: dict passed from view to template.

Output/Result: / shows home.html with title, message, and a list of 4 items. /hello/ shows 'Hello, World from Django!'

Practical 3: Django Models & Database (ORM)

Aim: Create database models using Django ORM and perform migrations.

1. Define Models (myapp/models.py):

```
from django.db import models

class Student(models.Model):
    # Field types
    name        = models.CharField(max_length=100)
    email       = models.EmailField(unique=True)
    roll_number = models.IntegerField()
    branch      = models.CharField(max_length=50)
    cgpa        = models.FloatField(default=0.0)
    is_placed   = models.BooleanField(default=False)
    joined_on   = models.DateField(auto_now_add=True)  # auto set on creation
    updated_at  = models.DateTimeField(auto_now=True)  # auto set on each save

    def __str__(self):
        return f"{self.name} ({self.roll_number})"

    class Meta:
        ordering = ['name']         # default sort by name
        verbose_name = 'Student'
        verbose_name_plural = 'Students'


class Company(models.Model):
    name     = models.CharField(max_length=200)
    website  = models.URLField(blank=True)
    location = models.CharField(max_length=100)
```

```
    def __str__(self):
        return self.name


class JobApplication(models.Model):
    STATUS_CHOICES = [
        ('applied',    'Applied'),
        ('shortlisted','Shortlisted'),
        ('rejected',   'Rejected'),
        ('selected',   'Selected'),
    ]
    student  = models.ForeignKey(Student, on_delete=models.CASCADE,
                                 related_name='applications')
    company  = models.ForeignKey(Company, on_delete=models.CASCADE)
    status   = models.CharField(max_length=20, choices=STATUS_CHOICES,
                                default='applied')
    applied_on = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.student.name} -> {self.company.name}"
```

2. Run Migrations:

```
# Create migration files (detect model changes)
python manage.py makemigrations

# Apply migrations to database
python manage.py migrate

# Check migration status
python manage.py showmigrations
```

3. ORM Queries (Django Shell):

```
# Open Django shell
python manage.py shell

# --- CREATE ---
s = Student.objects.create(name='Yug', email='yug@example.com',
                           roll_number=101, branch='IT')

# --- READ ---
Student.objects.all()             # all records
Student.objects.get(id=1)         # single record (raises error if not found)
Student.objects.filter(branch='IT')  # queryset
Student.objects.exclude(is_placed=True)
Student.objects.order_by('-cgpa') # descending
Student.objects.first()
Student.objects.count()

# --- UPDATE ---
s = Student.objects.get(id=1)
s.cgpa = 8.5
s.save()
# Bulk update:
Student.objects.filter(branch='IT').update(is_placed=True)

# --- DELETE ---
```

```
s = Student.objects.get(id=1)
s.delete()

# --- LOOKUPS ---
Student.objects.filter(cgpa__gte=7.0)         # cgpa >= 7.0
Student.objects.filter(name__icontains='yug') # case-insensitive contains
Student.objects.filter(branch__in=['IT','CE'])

# --- RELATIONSHIPS ---
student = Student.objects.get(id=1)
student.applications.all()    # all applications for this student
```

Explanation:

models.Model: base class for all models. Each class = one database table. Each field = one column. **makemigrations**: creates migration files from model changes. **migrate**: applies them to DB. **ForeignKey**: many-to-one relationship. **on_delete=CASCADE**: deletes related records. **__str__**: human-readable representation used in admin and shell.

Output/Result: makemigrations creates 0001_initial.py. migrate creates tables in db.sqlite3. ORM queries return QuerySets.

Practical 4: Django Admin Panel

Aim: Register models in Django Admin and manage data through the built-in admin interface.

1. Register Models (myapp/admin.py):

```
from django.contrib import admin
from .models import Student, Company, JobApplication

# Basic registration
admin.site.register(Company)

# Customized admin
@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display  = ('name', 'roll_number', 'branch', 'cgpa', 'is_placed')
    list_filter   = ('branch', 'is_placed')
    search_fields = ('name', 'email', 'roll_number')
    ordering      = ('-cgpa',)
    list_per_page = 20
    list_editable = ('is_placed',)   # edit directly in list view

    # Fieldsets organize the detail form
    fieldsets = (
        ('Personal Info', {
            'fields': ('name', 'email', 'roll_number')
        }),
        ('Academic Info', {
            'fields': ('branch', 'cgpa', 'is_placed')
        }),
    )


@admin.register(JobApplication)
class JobApplicationAdmin(admin.ModelAdmin):
    list_display  = ('student', 'company', 'status', 'applied_on')
```

```
    list_filter   = ('status', 'company')
    search_fields = ('student__name', 'company__name')
    date_hierarchy = 'applied_on'

# Customize admin site header
admin.site.site_header  = "College Placement Portal Admin"
admin.site.site_title   = "Placement Portal"
admin.site.index_title  = "Welcome to Admin Panel"
```

2. Create Superuser:

```
python manage.py createsuperuser

# Prompts:
# Username: admin
# Email: admin@example.com
# Password: *******

# Then run server and visit:
# http://127.0.0.1:8000/admin/
```

Explanation:

admin.site.register(Model): adds model to admin. **@admin.register**: decorator for customized admin class. **list_display**: columns shown in list view. **list_filter**: sidebar filters. **search_fields**: enables search box. **fieldsets**: organizes detail form into sections. **list_editable**: allows inline editing in list view.

Output/Result: Admin panel shows Student list with search, filter by branch/placed status, sortable columns. Site header shows 'College Placement Portal Admin'.

Practical 5: Django Forms

Aim: Create and process HTML forms using Django Forms, including validation.

1. Create Form (myapp/forms.py):

```
from django import forms
from .models import Student

# Django Form (not tied to model)
class ContactForm(forms.Form):
    name    = forms.CharField(max_length=100,
                              widget=forms.TextInput(attrs={
                                  'class': 'form-control',
                                  'placeholder': 'Enter your name'
                              }))
    email   = forms.EmailField(widget=forms.EmailInput(attrs={'class': 'form-control'}))
    message = forms.CharField(widget=forms.Textarea(attrs={
                                  'class': 'form-control', 'rows': 4
                              }))
    rating  = forms.ChoiceField(choices=[(i, i) for i in range(1, 6)])

    # Custom validation
    def clean_name(self):
        name = self.cleaned_data.get('name')
        if len(name) < 3:
            raise forms.ValidationError("Name must be at least 3 characters.")
        return name.title()   # capitalize

# ModelForm — auto-generates fields from model
```

```
class StudentForm(forms.ModelForm):
    class Meta:
        model  = Student
        fields = ['name', 'email', 'roll_number', 'branch', 'cgpa']
        widgets = {
            'name':  forms.TextInput(attrs={'class': 'form-control'}),
            'email': forms.EmailInput(attrs={'class': 'form-control'}),
            'cgpa':  forms.NumberInput(attrs={'class': 'form-control', 'step': '0.1'}),
        }
        labels = {
            'cgpa': 'CGPA (out of 10)',
        }
```

2. View to Handle Form (myapp/views.py):

```
from django.shortcuts import render, redirect
from django.contrib import messages
from .forms import StudentForm, ContactForm

def add_student(request):
    if request.method == 'POST':
        form = StudentForm(request.POST)
        if form.is_valid():
            form.save()                         # saves to DB (ModelForm)
            messages.success(request, 'Student added successfully!')
            return redirect('student_list')
        else:
            messages.error(request, 'Please fix the errors below.')
    else:
        form = StudentForm()   # blank form for GET

    return render(request, 'myapp/add_student.html', {'form': form})


def contact_view(request):
    if request.method == 'POST':
        form = ContactForm(request.POST)
        if form.is_valid():
            name    = form.cleaned_data['name']
            email   = form.cleaned_data['email']
            message = form.cleaned_data['message']
            # Process the data (e.g., send email)
            messages.success(request, f'Thanks {name}! We received your message.')
            return redirect('contact')
    else:
        form = ContactForm()
    return render(request, 'myapp/contact.html', {'form': form})
```

3. Template (myapp/templates/myapp/add_student.html):

```
{% load static %}
<!DOCTYPE html>
<html>
<head><title>Add Student</title></head>
<body>
  <h2>Add New Student</h2>

  {% if messages %}
```

```
    {% for msg in messages %}
      <div class="alert alert-{{ msg.tags }}">{{ msg }}</div>
    {% endfor %}
  {% endif %}

  <form method="POST" novalidate>
    {% csrf_token %}
    {{ form.as_p }}

    <!-- Or render manually: -->
    <!--
    <div>
      {{ form.name.label_tag }}
      {{ form.name }}
      {% if form.name.errors %}
        <span class="error">{{ form.name.errors }}</span>
      {% endif %}
    </div>
    -->

    <button type="submit">Add Student</button>
  </form>
</body>
</html>
```

Explanation:

forms.Form: standalone form with manual field definitions. **forms.ModelForm**: auto-generates fields from a model. **form.is_valid()**: runs all validators. **form.cleaned_data**: validated, Python-typed data. **form.save()**: saves ModelForm to DB. **{% csrf_token %}**: required CSRF protection token. **clean_fieldname()**: custom per-field validator.

Output/Result: GET request shows blank form. POST with valid data saves student and shows success message. Invalid data shows field errors inline.

Practical 6: Django Templates — Inheritance & Tags

Aim: Use Django Template Language (DTL) features: inheritance, tags, filters, and static files.

1. Base Template (templates/base.html):

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}My Site{% endblock %}</title>
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
    {% block extra_css %}{% endblock %}
</head>
<body>
    <!-- Navbar -->
    <nav>
        <a href="{% url 'home' %}">Home</a>
        <a href="{% url 'student_list' %}">Students</a>
        <a href="{% url 'add_student' %}">Add Student</a>
    </nav>

    <!-- Flash messages -->
```

```
    {% if messages %}
        {% for message in messages %}
            <div class="alert alert-{{ message.tags }}">{{ message }}</div>
        {% endfor %}
    {% endif %}

    <!-- Main content block -->
    <main>
        {% block content %}
        {% endblock %}
    </main>

    <footer>
        <p>&copy; 2024 College Placement Portal</p>
    </footer>

    <script src="{% static 'js/main.js' %}"></script>
    {% block extra_js %}{% endblock %}
</body>
</html>
```

2. Child Template (templates/myapp/home.html):

```
{% extends 'base.html' %}
{% load static %}

{% block title %}Home - Placement Portal{% endblock %}

{% block content %}
<h1>Welcome, {{ user.username|default:"Guest" }}!</h1>

<!-- if / elif / else -->
{% if students %}
    <p>Total students: {{ students|length }}</p>
    <ul>
    {% for student in students %}
        <li>
            {{ forloop.counter }}. {{ student.name|title }}
            — CGPA: {{ student.cgpa|floatformat:2 }}
            {% if student.is_placed %}
                <span style="color:green;">(Placed)</span>
            {% else %}
                <span style="color:red;">(Not Placed)</span>
            {% endif %}
        </li>
    {% empty %}
        <li>No students found.</li>
    {% endfor %}
    </ul>
{% else %}
    <p>No students in the database yet.</p>
{% endif %}

<!-- URL tag -->
<a href="{% url 'add_student' %}">Add New Student</a>

<!-- with tag — creates alias -->
```

```
{% with total=students|length %}
    <p>Showing {{ total }} record(s).</p>
{% endwith %}
{% endblock %}
```

3. Common Template Filters:

```
{{ name|upper }}              {# UPPERCASE #}
{{ name|lower }}              {# lowercase #}
{{ name|title }}              {# Title Case #}
{{ name|truncatewords:5 }}    {# First 5 words #}
{{ name|truncatechars:20 }}   {# First 20 chars #}
{{ cgpa|floatformat:2 }}      {# 2 decimal places #}
{{ date|date:"d M Y" }}       {# 25 Jan 2024 #}
{{ count|default:"None" }}    {# fallback if empty #}
{{ items|length }}            {# count items #}
{{ items|first }}             {# first element #}
{{ items|last }}              {# last element #}
{{ text|linebreaks }}         {#
 to <br> #}
{{ text|safe }}               {# render HTML #}
```

4. Static Files Setup (settings.py):

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']  # where you put your files

# In production:
STATIC_ROOT = BASE_DIR / 'staticfiles'
# Run: python manage.py collectstatic
```

Explanation:

{% extends %}: inherit from base template. **{% block %}**: defines replaceable sections. **{% include %}**: embeds another template. **{% url 'name' %}**: generates URL by name. **{% static 'path' %}**: generates static file URL. **{% load %}**: loads template tag library. **{% for %} {% empty %}**: loop with fallback. **forloop.counter**: loop index (1-based).

> **Output/Result:** Child templates extend base.html, inheriting navbar and footer. Content block is replaced. {% for %} renders student list with conditional placed/not-placed badge.

Practical 7: CRUD Operations — Student Management

Aim: Implement full Create, Read, Update, Delete (CRUD) functionality for a Student model.

views.py — All CRUD Views:

```
from django.shortcuts import render, get_object_or_404, redirect
from django.contrib import messages
from .models import Student
from .forms import StudentForm

# READ — List all students
def student_list(request):
    students = Student.objects.all().order_by('name')
    return render(request, 'myapp/student_list.html', {'students': students})

# READ — Single student detail
def student_detail(request, pk):
    student = get_object_or_404(Student, pk=pk)
    return render(request, 'myapp/student_detail.html', {'student': student})
```

```
# CREATE — Add new student
def student_create(request):
    if request.method == 'POST':
        form = StudentForm(request.POST)
        if form.is_valid():
            student = form.save()
            messages.success(request, f'{student.name} added successfully!')
            return redirect('student_list')
    else:
        form = StudentForm()
    return render(request, 'myapp/student_form.html', {'form': form, 'action': 'Add'})

# UPDATE — Edit existing student
def student_update(request, pk):
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        form = StudentForm(request.POST, instance=student)  # bind to existing object
        if form.is_valid():
            form.save()
            messages.success(request, 'Student updated successfully!')
            return redirect('student_detail', pk=student.pk)
    else:
        form = StudentForm(instance=student)   # pre-fill form with existing data
    return render(request, 'myapp/student_form.html', {'form': form, 'action': 'Update'})

# DELETE — Remove student
def student_delete(request, pk):
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        name = student.name
        student.delete()
        messages.success(request, f'{name} deleted successfully!')
        return redirect('student_list')
    return render(request, 'myapp/student_confirm_delete.html', {'student': student})
```

urls.py — URL Patterns:

```
from django.urls import path
from . import views

urlpatterns = [
    path('students/',              views.student_list,   name='student_list'),
    path('students/<int:pk>/',     views.student_detail, name='student_detail'),
    path('students/add/',          views.student_create, name='student_create'),
    path('students/<int:pk>/edit/',views.student_update, name='student_update'),
    path('students/<int:pk>/del/', views.student_delete, name='student_delete'),
]
```

student_list.html (partial):

```
{% for student in students %}
<tr>
  <td>{{ student.name }}</td>
  <td>{{ student.branch }}</td>
  <td>{{ student.cgpa }}</td>
  <td>
    <a href="{% url 'student_update' student.pk %}">Edit</a>
    <a href="{% url 'student_delete' student.pk %}"
```

```
        onclick="return confirm('Delete {{ student.name }}?')">Delete</a>
    </td>
  </tr>
  {% endfor %}
```

Explanation:

get_object_or_404(): gets object or returns 404 response — safer than .get(). **instance=student**: binds ModelForm to existing record for update. **form.save()**: INSERT for new, UPDATE for existing (when instance provided). **redirect()**: Post-Redirect-Get pattern prevents double form submission. **<int:pk>**: URL converter that captures integer primary key.

Output/Result: List page shows all students. Edit pre-fills the form. Delete shows confirmation page. Each operation shows a success message.

Practical 8: Django User Authentication

Aim: Implement user registration, login, and logout using Django's built-in authentication system.

1. views.py — Auth Views:

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from django.contrib.auth.decorators import login_required
from django.contrib import messages

# Registration
def register_view(request):
    if request.method == 'POST':
        form = UserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)    # auto-login after register
            messages.success(request, f'Account created for {user.username}!')
            return redirect('home')
    else:
        form = UserCreationForm()
    return render(request, 'auth/register.html', {'form': form})

# Login
def login_view(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(request, username=username, password=password)
            if user is not None:
                login(request, user)
                messages.success(request, f'Welcome back, {username}!')
                return redirect('home')
            else:
                messages.error(request, 'Invalid username or password.')
    else:
        form = AuthenticationForm()
    return render(request, 'auth/login.html', {'form': form})
```

```
# Logout
def logout_view(request):
    logout(request)
    messages.info(request, 'You have been logged out.')
    return redirect('login')

# Protected view — requires login
@login_required(login_url='/login/')
def dashboard(request):
    return render(request, 'myapp/dashboard.html', {'user': request.user})
```

2. urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('register/', views.register_view, name='register'),
    path('login/',    views.login_view,    name='login'),
    path('logout/',   views.logout_view,   name='logout'),
    path('dashboard/',views.dashboard,     name='dashboard'),
]
```

3. Template Tags for Auth (in any template):

```
{% if user.is_authenticated %}
    <p>Hello, {{ user.username }}!</p>
    <a href="{% url 'logout' %}">Logout</a>
    <a href="{% url 'dashboard' %}">Dashboard</a>
{% else %}
    <a href="{% url 'login' %}">Login</a>
    <a href="{% url 'register' %}">Register</a>
{% endif %}
```

4. settings.py — Auth Settings:

```
LOGIN_URL          = '/login/'
LOGIN_REDIRECT_URL = '/'
LOGOUT_REDIRECT_URL = '/login/'
```

Explanation:

authenticate(): checks username/password, returns user or None. **login()**: creates a session for the user. **logout()**: destroys the session. **@login_required**: decorator that redirects unauthenticated users. **UserCreationForm**: built-in form with username/password/confirm fields. **request.user**: current user object (AnonymousUser if not logged in). **user.is_authenticated**: True/False.

> **Output/Result:** Unregistered users can register. Login with credentials creates a session. @login_required redirects to /login/ if not authenticated. Dashboard shows personalized welcome.

Practical 9: Django Class-Based Views (CBVs)

Aim: Use Django's class-based generic views (ListView, DetailView, CreateView, UpdateView, DeleteView).

views.py — Class-Based Views:

```
from django.views.generic import (ListView, DetailView,
                                  CreateView, UpdateView, DeleteView)
from django.urls import reverse_lazy
from django.contrib.auth.mixins import LoginRequiredMixin
from .models import Student
```

```
from .forms import StudentForm

# List all students
class StudentListView(LoginRequiredMixin, ListView):
    model               = Student
    template_name       = 'myapp/student_list.html'
    context_object_name = 'students'   # name used in template
    paginate_by         = 10           # auto-pagination

    def get_queryset(self):
        # Custom queryset with search
        query = self.request.GET.get('q', '')
        if query:
            return Student.objects.filter(name__icontains=query)
        return Student.objects.all().order_by('name')

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['search_query'] = self.request.GET.get('q', '')
        return context


# Show single student
class StudentDetailView(DetailView):
    model         = Student
    template_name = 'myapp/student_detail.html'


# Create new student
class StudentCreateView(LoginRequiredMixin, CreateView):
    model         = Student
    form_class    = StudentForm
    template_name = 'myapp/student_form.html'
    success_url   = reverse_lazy('student_list')

    def form_valid(self, form):
        # Called when form is valid — before saving
        from django.contrib import messages
        messages.success(self.request, 'Student created!')
        return super().form_valid(form)


# Update student
class StudentUpdateView(LoginRequiredMixin, UpdateView):
    model         = Student
    form_class    = StudentForm
    template_name = 'myapp/student_form.html'
    success_url   = reverse_lazy('student_list')


# Delete student
class StudentDeleteView(LoginRequiredMixin, DeleteView):
    model         = Student
    template_name = 'myapp/student_confirm_delete.html'
    success_url   = reverse_lazy('student_list')
```

urls.py with CBVs:

```
from django.urls import path
from .views import (StudentListView, StudentDetailView,
                    StudentCreateView, StudentUpdateView, StudentDeleteView)

urlpatterns = [
    path('',                    StudentListView.as_view(),   name='student_list'),
    path('<int:pk>/',           StudentDetailView.as_view(), name='student_detail'),
    path('add/',                StudentCreateView.as_view(), name='student_create'),
    path('<int:pk>/edit/',      StudentUpdateView.as_view(), name='student_update'),
    path('<int:pk>/delete/',    StudentDeleteView.as_view(), name='student_delete'),
]
```

Explanation:

ListView: renders a list of objects. **DetailView**: renders one object. **CreateView / UpdateView**: render and process forms. **DeleteView**: confirms and deletes. **LoginRequiredMixin**: restricts access to logged-in users (use before other mixins). **reverse_lazy()**: lazy URL resolution needed in class attributes. **get_queryset()**: override to customize data. **get_context_data()**: add extra context. **paginate_by**: auto-adds pagination with page_obj in template.

Output/Result: Same CRUD functionality as FBVs but with less boilerplate. Pagination auto-adds Next/Prev controls. Search filters the queryset.

Practical 10: Django REST API with JsonResponse

Aim: Build a simple REST-like API using Django's JsonResponse and handle JSON requests.

views.py — API Views:

```
import json
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_http_methods
from .models import Student

# GET all students as JSON
def api_students(request):
    students = Student.objects.values(
        'id', 'name', 'email', 'branch', 'cgpa', 'is_placed'
    )
    data = {
        'count':    students.count(),
        'students': list(students),
        'status':   'success'
    }
    return JsonResponse(data)

# GET single student
def api_student_detail(request, pk):
    try:
        student = Student.objects.get(pk=pk)
        data = {
            'id':         student.id,
            'name':       student.name,
            'email':      student.email,
            'branch':     student.branch,
            'cgpa':       student.cgpa,
```

```
            'is_placed':  student.is_placed,
        }
        return JsonResponse({'student': data, 'status': 'success'})
    except Student.DoesNotExist:
        return JsonResponse({'error': 'Student not found', 'status': 'error'}, status=404)

# POST — Create student via API
@csrf_exempt
@require_http_methods(["POST"])
def api_create_student(request):
    try:
        data    = json.loads(request.body)
        student = Student.objects.create(
            name        = data['name'],
            email       = data['email'],
            roll_number = data['roll_number'],
            branch      = data['branch'],
            cgpa        = data.get('cgpa', 0.0),
        )
        return JsonResponse({
            'id':      student.id,
            'message': f'{student.name} created.',
            'status':  'success'
        }, status=201)
    except KeyError as e:
        return JsonResponse({'error': f'Missing field: {e}', 'status': 'error'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e), 'status': 'error'}, status=500)

# DELETE
@csrf_exempt
def api_delete_student(request, pk):
    if request.method == 'DELETE':
        try:
            student = Student.objects.get(pk=pk)
            student.delete()
            return JsonResponse({'message': 'Deleted.', 'status': 'success'})
        except Student.DoesNotExist:
            return JsonResponse({'error': 'Not found'}, status=404)
```

urls.py:

```
path('api/students/',          views.api_students,        name='api_students'),
path('api/students/<int:pk>/', views.api_student_detail,  name='api_student_detail'),
path('api/students/create/',   views.api_create_student,  name='api_create_student'),
path('api/students/<int:pk>/delete/', views.api_delete_student, name='api_delete_student'),
```

Test with curl:

```
# GET all students
curl http://127.0.0.1:8000/api/students/

# POST new student
curl -X POST http://127.0.0.1:8000/api/students/create/ \
     -H "Content-Type: application/json" \
     -d '{"name":"Yug","email":"yug@test.com","roll_number":101,"branch":"IT"}'

# DELETE
```

```
curl -X DELETE http://127.0.0.1:8000/api/students/1/delete/
```

Explanation:

JsonResponse: returns HTTP response with JSON content-type. Accepts a dict. **.values()**: returns QuerySet of dicts (not model objects). **@csrf_exempt**: disables CSRF check for API (use carefully). **json.loads(request.body)**: parses JSON request body. **status=**: HTTP status code (200, 201, 400, 404, 500). For production APIs, use Django REST Framework (DRF).

Output/Result: GET /api/students/ returns JSON with student list and count. POST creates a new record and returns 201. Invalid data returns 400 with error message.

Practical 11: Django Signals & Middleware (Concept + Example)

Aim: Understand and implement Django signals (post_save) and basic custom middleware.

1. Signals (myapp/signals.py):

```
from django.db.models.signals import post_save, pre_delete
from django.dispatch import receiver
from django.contrib.auth.models import User
from .models import Student

# Signal fired after a Student is saved
@receiver(post_save, sender=Student)
def student_saved(sender, instance, created, **kwargs):
    if created:
        print(f"[SIGNAL] New student created: {instance.name}")
        # Could send welcome email, create profile, log activity, etc.
    else:
        print(f"[SIGNAL] Student updated: {instance.name}")

# Signal fired before Student is deleted
@receiver(pre_delete, sender=Student)
def student_deleting(sender, instance, **kwargs):
    print(f"[SIGNAL] About to delete: {instance.name}")

# Create a profile automatically when a User registers
@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        # Could create a Profile model linked to User
        print(f"[SIGNAL] New user registered: {instance.username}")
```

Register Signals in apps.py:

```
# myapp/apps.py
from django.apps import AppConfig

class MyappConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'myapp'

    def ready(self):
        import myapp.signals   # connect signals when app loads
```

2. Custom Middleware (myapp/middleware.py):

```
import time
import logging
```

```
logger = logging.getLogger(__name__)

class RequestTimingMiddleware:
    """Logs the time taken for each request."""

    def __init__(self, get_response):
        self.get_response = get_response   # next middleware or view

    def __call__(self, request):
        start = time.time()

        # Code before the view
        print(f"[MIDDLEWARE] Incoming: {request.method} {request.path}")

        response = self.get_response(request)   # call the view

        # Code after the view
        duration = (time.time() - start) * 1000
        print(f"[MIDDLEWARE] Response: {response.status_code} in {duration:.2f}ms")

        # Add custom header to response
        response['X-Response-Time'] = f"{duration:.2f}ms"

        return response


class BlockIPMiddleware:
    """Blocks specific IP addresses."""
    BLOCKED_IPS = ['192.168.1.100']

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        ip = request.META.get('REMOTE_ADDR')
        if ip in self.BLOCKED_IPS:
            from django.http import HttpResponseForbidden
            return HttpResponseForbidden('Access denied.')
        return self.get_response(request)
```

Register Middleware in settings.py:

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    ...
    'myapp.middleware.RequestTimingMiddleware',   # add here
    'myapp.middleware.BlockIPMiddleware',
]
```

Explanation:

Signals: decoupled event system. **post_save**: fires after model .save(). **pre_delete**: fires before model .delete(). **created**: True if new object, False if update. **Middleware**: processes every request/response. **__init__** receives get_response (next in chain). **__call__** wraps the view: code before = request processing, code after = response processing.

Output/Result: Every Student save prints signal message. Middleware logs request path and duration. Custom X-Response-Time header appears in browser DevTools Network tab.

Practical 12: Complete Mini Project — College Placement Portal

Aim: Build a mini Placement Portal combining models, views, forms, templates, and authentication.

Project Structure:

```
placement_portal/
■■■ manage.py
■■■ placement_portal/
■   ■■■ settings.py
■   ■■■ urls.py
■■■ portal/
    ■■■ models.py
    ■■■ views.py
    ■■■ forms.py
    ■■■ urls.py
    ■■■ admin.py
    ■■■ templates/portal/
        ■■■ base.html
        ■■■ home.html
        ■■■ job_list.html
        ■■■ job_detail.html
        ■■■ apply.html
```

models.py:

```
from django.db import models
from django.contrib.auth.models import User

class Job(models.Model):
    title       = models.CharField(max_length=200)
    company     = models.CharField(max_length=200)
    description = models.TextField()
    salary      = models.DecimalField(max_digits=10, decimal_places=2)
    location    = models.CharField(max_length=100)
    skills      = models.CharField(max_length=300)
    deadline    = models.DateField()
    posted_on   = models.DateTimeField(auto_now_add=True)
    is_active   = models.BooleanField(default=True)

    def __str__(self): return f"{self.title} at {self.company}"

class Application(models.Model):
    STATUS = [('P','Pending'),('S','Shortlisted'),('R','Rejected')]
    job        = models.ForeignKey(Job, on_delete=models.CASCADE,
                                   related_name='applications')
    applicant  = models.ForeignKey(User, on_delete=models.CASCADE)
    cover_note = models.TextField(blank=True)
    status     = models.CharField(max_length=1, choices=STATUS, default='P')
    applied_on = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('job', 'applicant')   # one application per job

    def __str__(self): return f"{self.applicant} -> {self.job}"
```

views.py (key views):

```
from django.shortcuts import render, get_object_or_404, redirect
```

```
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from .models import Job, Application

def job_list(request):
    jobs = Job.objects.filter(is_active=True).order_by('-posted_on')
    search = request.GET.get('q', '')
    if search:
        jobs = jobs.filter(title__icontains=search) |               jobs.filter(company__icontains=search)
    return render(request, 'portal/job_list.html', {'jobs': jobs, 'search': search})

def job_detail(request, pk):
    job = get_object_or_404(Job, pk=pk, is_active=True)
    already_applied = False
    if request.user.is_authenticated:
        already_applied = Application.objects.filter(
            job=job, applicant=request.user).exists()
    return render(request, 'portal/job_detail.html',
                  {'job': job, 'already_applied': already_applied})

@login_required
def apply_job(request, pk):
    job = get_object_or_404(Job, pk=pk, is_active=True)
    if Application.objects.filter(job=job, applicant=request.user).exists():
        messages.warning(request, 'You have already applied for this job.')
        return redirect('job_detail', pk=pk)
    if request.method == 'POST':
        Application.objects.create(
            job=job,
            applicant=request.user,
            cover_note=request.POST.get('cover_note', '')
        )
        messages.success(request, f'Applied to {job.company} successfully!')
        return redirect('job_list')
    return render(request, 'portal/apply.html', {'job': job})

@login_required
def my_applications(request):
    apps = Application.objects.filter(
        applicant=request.user).select_related('job').order_by('-applied_on')
    return render(request, 'portal/my_applications.html', {'applications': apps})
```

Key Concepts Used:

This project combines: **ForeignKey** relationships (Job-Application-User), **unique_together** constraint to prevent duplicate applications, **Q object** OR-filter for search (|), **select_related()** to avoid N+1 queries, **.exists()** for efficient boolean check, **@login_required** protection, and **messages** framework for user feedback.

Output/Result: Home shows active jobs with search. Job detail shows Apply button (disabled if already applied). My Applications shows status of all applied jobs. Admin can manage jobs and update application status.

End of Django Practicals — Best of luck! All the best for GTU exams!